School of Humanities & Sciences
D Y Patil Deemed to be University
Navi Mumbai

Machine Learning Project

Course Code: 2522MCC2P2
Course Name: Machine Learning

Submitted by

DY25SHSP0MCA159
Rutvik Suryakant Varankar

in partial fulfilment for the award of the degree

of

M.C.A. Semester II (2025-2026)

TABLE OF CONTENT

SR No	TOPIC	Page No
1	Introduction	3
2	Scope and objectives	3
3	Project description	4
4	Existing system	5
5	Proposed system	5
6	Technology used	6
7	Implementation of project	7
8	Screenshots of user interface or output	18
9	Conclusion	20

1. Introduction

Sentiment Analysis is a technique in Natural Language Processing (NLP) that is used to identify and classify opinions expressed in textual data. With the rapid growth of digital platforms, people frequently share their views through movie reviews, social media posts, and online feedback. Analyzing such a large volume of data manually is time-consuming and inefficient, which creates a need for automated systems that can process and interpret this information quickly and accurately. This project focuses on developing a machine learning-based system that analyzes movie reviews and determines whether they express a positive or negative sentiment.

The system uses text preprocessing techniques to clean and prepare the data, followed by feature extraction using TF-IDF to convert text into numerical form. A classification algorithm, namely Logistic Regression, is then applied to predict the sentiment of the reviews. The model is trained on a labeled dataset and evaluated to ensure reliable performance. In addition to the backend model, a user-friendly frontend interface has been developed using Flask, where users can enter their own movie reviews and instantly receive sentiment predictions. This integration of machine learning with a web interface makes the system interactive, practical, and easy to use.

The main objective of this project is to demonstrate how machine learning and NLP techniques can be applied to solve real-world problems involving text data. By automating sentiment classification and providing an accessible interface, the system helps in understanding user opinions efficiently and can be extended to applications such as product review analysis, customer feedback systems, and social media monitoring.

2. Scope and objectives

The scope of this project lies in developing an automated system for analyzing and classifying movie reviews based on their sentiment using machine learning and Natural Language Processing techniques. The system is designed to handle textual data efficiently and can be applied not only to movie reviews but also extended to other domains such as product reviews, customer feedback, and social media analysis. With the integration of a Flask-based frontend, the project provides a user-friendly interface where users can input their own reviews and receive real-time sentiment predictions. This makes the system practical, scalable, and suitable for real-world applications that require quick interpretation of large volumes of opinion-based data.

The main objectives of this project are to preprocess and clean raw textual data, convert it into meaningful numerical representations using TF-IDF, and build an accurate classification model using Logistic Regression. Another key objective is to evaluate the model's performance using appropriate metrics and ensure reliable predictions. Additionally, the project aims to integrate the trained model with a web-based interface using Flask, enabling user interaction and real-time analysis. Overall, the objective is to demonstrate the effective use of machine learning and NLP in solving real-world problems related to text analysis while creating a system that is efficient, accurate, and easy to use.

3. Project description

The Sentiment Analysis project is designed to develop an intelligent and automated system that classifies movie reviews as positive or negative using machine learning and Natural Language Processing (NLP) techniques. The project begins with the collection of a labeled dataset consisting of thousands of movie reviews, each tagged with its corresponding sentiment. This dataset serves as the foundation for training and evaluating the model. Since raw textual data often contains noise and inconsistencies, an essential step in the project is data preprocessing. This involves cleaning the text by removing HTML tags, punctuation, numbers, and unnecessary characters, as well as converting all text into lowercase to maintain uniformity. These preprocessing steps help in improving the quality of the data and ensure better model performance.

After cleaning the data, the next step involves feature extraction, where textual data is transformed into a numerical format that can be understood by machine learning algorithms. In this project, the TF-IDF (Term Frequency–Inverse Document Frequency) technique is used to measure the importance of words in a review relative to the entire dataset. This approach helps in capturing the significance of words that contribute most to determining sentiment. Once the features are extracted, the dataset is divided into training and testing sets to build and evaluate the model effectively.

A Logistic Regression algorithm is then applied to train the sentiment classification model. This algorithm is chosen due to its efficiency, simplicity, and strong performance in text classification tasks. The trained model learns patterns and relationships between words and their associated sentiments. After training, the model is evaluated using the test dataset to measure its accuracy and reliability. Performance metrics such as accuracy are used to

determine how well the model predicts unseen data.

To enhance usability and make the system more interactive, a web-based interface is developed using Flask. This frontend allows users to input their own movie reviews through a simple interface, and the system processes the input in real time to predict the sentiment. The integration of the trained machine learning model with a Flask application demonstrates the deployment aspect of the project, making it accessible to users without requiring technical knowledge. Overall, the project presents a complete end-to-end pipeline, including data collection, preprocessing, feature extraction, model training, evaluation, and deployment. It highlights how machine learning and NLP can be effectively combined to solve real-world problems related to text analysis and opinion mining.

4. Existing system

In traditional approaches, sentiment analysis is often performed manually, where individuals read and interpret reviews to determine whether they are positive or negative. While this may work for small datasets, it becomes time-consuming and inefficient when dealing with large volumes of data. Manual analysis is also prone to human bias and inconsistency, making it less reliable for accurate results.

Some existing automated systems use rule-based or keyword-based techniques, where predefined positive and negative words are used for classification. However, these systems fail to understand context, sarcasm, and complex sentence structures, which can lead to incorrect predictions. They also require constant updates and are not scalable for dynamic data. Due to these limitations, existing systems lack accuracy and efficiency, highlighting the need for more advanced machine learning-based approaches.

5. Proposed system

The proposed system aims to develop an automated and efficient sentiment analysis model using machine learning and Natural Language Processing techniques. Unlike traditional methods, this system processes large volumes of textual data quickly and accurately without human intervention. It begins by preprocessing the input text to remove noise such as punctuation, HTML tags, and unnecessary characters, ensuring that the data is clean and consistent. The cleaned text is then transformed into numerical form using the TF-IDF technique, which helps in identifying the importance of words in determining sentiment.

A Logistic Regression model is trained on the processed dataset to learn patterns associated with positive and negative reviews. Once trained, the model is capable of predicting the sentiment of new, unseen reviews with good accuracy. To make the system more interactive and user-friendly, a web-based interface is developed using Flask, where users can enter their own movie reviews and instantly receive sentiment predictions. This integration of machine learning with a web application allows real-time analysis and enhances usability. Overall, the proposed system provides a scalable, accurate, and efficient solution for sentiment classification, overcoming the limitations of existing manual and rule-based approaches.

6. Technology used

- **Programming Language:**
  a. **Python**

- **Libraries:**
  a. **Pandas –** for data manipulation and dataset handling
  b. NumPy – for numerical operations
  c. **Scikit-learn –** for machine learning (TF-IDF, Logistic Regression, model evaluation)
  d. Joblib – for saving and loading the trained model

- **Techniques:**
  a. Natural Language Processing (NLP)
  b. TF-IDF (Term Frequency–Inverse Document Frequency)
  c. Machine Learning Classification

- **Framework:**
  a. Flask – for developing the web-based user interface

- **Tools/Environment:**
  a. Jupyter Notebook / VS Code (for development and testing)

7. Implementation of project (code file name followed by code)

- **task.ipynb**

1. Import All Required Libraries

```
pip install pandas numpy matplotlib seaborn scikit-learn nltk joblib
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import joblib
import warnings
warnings.filterwarnings('ignore')

# Sklearn - Preprocessing & Splitting
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV, StratifiedKFold, learning_curve
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder

# Sklearn - Models
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, VotingClassifier

# Sklearn - Metrics
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    roc_auc_score, roc_curve, precision_recall_curve, auc
)

# NLTK for text preprocessing
import nltk
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

print('All libraries imported successfully!')
```

2. Load Dataset

```
df = pd.read_csv('IMDB Dataset.csv')
print('Shape:', df.shape)
print('\nClass Distribution:')
print(df['sentiment'].value_counts())
df.head(5)
```

3. Text Cleaning & Preprocessing

```
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    """Full text cleaning pipeline."""
    # 1. Remove HTML tags (common in IMDB reviews)
    text = re.sub(r'<.*?>', ' ', text)
    # 2. Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)
    # 3. Lowercase
    text = text.lower()
    # 4. Remove punctuation and numbers
    text = re.sub(r'[^a-z\s]', '', text)
    # 5. Tokenize
    tokens = text.split()
    # 6. Remove stop words and lemmatize
    tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words and len(w) > 2]
    return ' '.join(tokens)

print('Cleaning text... (may take a minute)')
df['clean_review'] = df['review'].apply(clean_text)

# Show before/after
print('\nOriginal:')
print(df['review'].iloc[0][:300])
print('\nCleaned:')
print(df['clean_review'].iloc[0][:300])
```

4. Feature Engineering - Review Length & Extra Features

```
# Add numeric features
df['review_length']     = df['review'].apply(len)
df['word_count']        = df['review'].apply(lambda x: len(x.split()))
df['exclamation_count'] = df['review'].apply(lambda x: x.count('!'))
df['question_count']    = df['review'].apply(lambda x: x.count('?'))

# Visualise length distribution by sentiment
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
for sentiment, color in [('positive', 'steelblue'), ('negative', 'tomato')]:
    subset = df[df['sentiment'] == sentiment]
    axes[0].hist(subset['word_count'], bins=50, alpha=0.6, label=sentiment, color=color)
    axes[1].hist(subset['exclamation_count'], bins=30, alpha=0.6, label=sentiment, color=color)
axes[0].set_title('Word Count by Sentiment'); axes[0].legend()
axes[1].set_title('Exclamation Marks by Sentiment'); axes[1].legend()
plt.tight_layout()
plt.show()
```

5. Encode Labels & Split Data

```
# Encode labels: positive=1, negative=0
le = LabelEncoder()
df['label'] = le.fit_transform(df['sentiment'])  # negative=0, positive=1
print('Label mapping:', dict(zip(le.classes_, le.transform(le.classes_))))

X = df['clean_review']
y = df['label']

# Stratified split to preserve class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)
print(f'Train size: {len(X_train)} | Test size: {len(X_test)}')
print(f'Train class balance: {y_train.value_counts().to_dict()}')
```

6. TF-IDF Vectorization

```
vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=50000,
    ngram_range=(1, 2),       # unigrams + bigrams
    sublinear_tf=True,        # apply log normalization to TF
    min_df=2                  # ignore very rare terms
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf  = vectorizer.transform(X_test)

print('Train TF-IDF shape:', X_train_tfidf.shape)
print('Test  TF-IDF shape:', X_test_tfidf.shape)
```

7. Train Multiple Models & Compare

```
models = {
    'Logistic Regression': LogisticRegression(class_weight='balanced', max_iter=1000, C=1),
    'Naive Bayes':         MultinomialNB(alpha=0.1),
    'LinearSVC':           LinearSVC(class_weight='balanced', max_iter=2000, C=1),
    'Random Forest':       RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42, n_jobs=-1),
}

results = {}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print(f'{"Model":<25} {"CV Mean":>10} {"CV Std":>10} {"Test Acc":>10}')
print('-' * 60)
for name, model in models.items():
    cv_scores = cross_val_score(model, X_train_tfidf, y_train, cv=skf, scoring='accuracy', n_jobs=-1)
    model.fit(X_train_tfidf, y_train)
    test_acc = accuracy_score(y_test, model.predict(X_test_tfidf))
    results[name] = {'model': model, 'cv_mean': cv_scores.mean(), 'cv_std': cv_scores.std(), 'test_acc': test_acc}
    print(f'{name:<25} {cv_scores.mean():>10.4f} {cv_scores.std():>10.4f} {test_acc:>10.4f}')
```

8. Visualise Model Comparison

```
names     = list(results.keys())
cv_means  = [results[n]['cv_mean'] for n in names]
cv_stds   = [results[n]['cv_std']  for n in names]
test_accs = [results[n]['test_acc'] for n in names]

x = np.arange(len(names))
width = 0.35
fig, ax = plt.subplots(figsize=(12, 5))
bars1 = ax.bar(x - width/2, cv_means, width, label='CV Accuracy', color='steelblue', yerr=cv_stds, capsize=4)
bars2 = ax.bar(x + width/2, test_accs, width, label='Test Accuracy', color='tomato')
ax.set_xticks(x); ax.set_xticklabels(names, rotation=15)
ax.set_ylim(0.8, 1.0)
ax.set_ylabel('Accuracy'); ax.set_title('Model Comparison')
ax.legend()
for bar in bars1: ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002, f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)
for bar in bars2: ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002, f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.show()
```

9. Hyperparameter Tuning (Best Model - Logistic Regression)

```
param_grid = {
    'C':       [0.01, 0.1, 1, 10],
    'penalty': ['l2'],
    'solver':  ['liblinear']
}
grid = GridSearchCV(
    LogisticRegression(class_weight='balanced', max_iter=1000),
    param_grid, cv=skf, scoring='accuracy', n_jobs=-1, verbose=1
)
grid.fit(X_train_tfidf, y_train)
best_model = grid.best_estimator_

print('Best Params:', grid.best_params_)
print('Best CV Score:', round(grid.best_score_, 4))
```

10. Voting Classifier (Ensemble)

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr',  LogisticRegression(class_weight='balanced', max_iter=1000, C=1)),
        ('svm', LinearSVC(class_weight='balanced', max_iter=2000, C=1)),
    ],
    voting='hard'
)
voting_clf.fit(X_train_tfidf, y_train)
voting_acc = accuracy_score(y_test, voting_clf.predict(X_test_tfidf))
print(f'Voting Classifier Test Accuracy: {voting_acc:.4f}')
```

11. Detailed Evaluation of Best Model

```
y_pred      = best_model.predict(X_test_tfidf)
y_pred_prob = best_model.predict_proba(X_test_tfidf)[:, 1]

print('=== Classification Report ===')
print(classification_report(y_test, y_pred, target_names=['Negative', 'Positive']))
print(f'ROC-AUC Score: {roc_auc_score(y_test, y_pred_prob):.4f}')
```

12. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted'); plt.ylabel('Actual')
plt.title('Confusion Matrix - Best Model')
plt.tight_layout()
plt.show()
```

13. ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
roc_auc     = auc(fpr, tpr)

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, color='steelblue', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Classifier')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve'); plt.legend()
plt.tight_layout()
plt.show()
```

14. Precision-Recall Curve

```
precision, recall, _ = precision_recall_curve(y_test, y_pred_prob)
pr_auc = auc(recall, precision)

plt.figure(figsize=(7, 5))
plt.plot(recall, precision, color='tomato', lw=2, label=f'PR curve (AUC = {pr_auc:.4f})')
plt.xlabel('Recall'); plt.ylabel('Precision')
plt.title('Precision-Recall Curve'); plt.legend()
plt.tight_layout()
plt.show()
```

15. Learning Curves (Overfitting Check)

```
train_sizes, train_scores, val_scores = learning_curve(
    best_model, X_train_tfidf, y_train,
    cv=5, scoring='accuracy', n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 8)
)

plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='steelblue', label='Training Accuracy')
plt.fill_between(train_sizes,
                 train_scores.mean(axis=1) - train_scores.std(axis=1),
                 train_scores.mean(axis=1) + train_scores.std(axis=1), alpha=0.1, color='steelblue')
plt.plot(train_sizes, val_scores.mean(axis=1), 'o-', color='tomato', label='Validation Accuracy')
plt.fill_between(train_sizes,
                 val_scores.mean(axis=1) - val_scores.std(axis=1),
                 val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=0.1, color='tomato')
plt.xlabel('Training Size'); plt.ylabel('Accuracy')
plt.title('Learning Curves'); plt.legend()
plt.tight_layout()
plt.show()
```

16. Feature Importance — Top Words for Each Sentiment

```
feature_names = np.array(vectorizer.get_feature_names_out())
coef = best_model.coef_[0]
top_n = 20

top_positive_idx = coef.argsort()[-top_n:][::-1]
top_negative_idx = coef.argsort()[:top_n]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Positive features
axes[0].barh(feature_names[top_positive_idx][::-1], coef[top_positive_idx][::-1], color='steelblue')
axes[0].set_title('Top 20 Words → POSITIVE Sentiment')
axes[0].set_xlabel('Coefficient')

# Negative features
axes[1].barh(feature_names[top_negative_idx], np.abs(coef[top_negative_idx]), color='tomato')
axes[1].set_title('Top 20 Words → NEGATIVE Sentiment')
axes[1].set_xlabel('|Coefficient|')

plt.tight_layout()
plt.show()
```

17. Save Model & Vectorizer

```
joblib.dump(best_model, 'sentiment_model.pkl')
joblib.dump(vectorizer, 'tfidf_vectorizer.pkl')
joblib.dump(le, 'label_encoder.pkl')
print('Model, vectorizer, and label encoder saved!')
```

18. Reusable Prediction Function

```python
def predict_sentiment(text, model=best_model, vec=vectorizer, encoder=le):
    """
    Predict sentiment for a raw review string.
    Returns: label string and confidence score.
    """
    cleaned   = clean_text(text)
    tfidf_vec = vec.transform([cleaned])
    pred      = model.predict(tfidf_vec)[0]
    prob      = model.predict_proba(tfidf_vec)[0][pred]
    label     = encoder.inverse_transform([pred])[0]
    return label, round(prob * 100, 2)

# --- Test it ---
test_reviews = [
    "This movie was absolutely fantastic! One of the best films I've ever seen.",
    "Terrible film. Waste of time. The acting was horrible and the plot made no sense.",
    "It was an okay movie. Nothing special but not terrible either."
]

print('=== Prediction Examples ===')
for review in test_reviews:
    label, confidence = predict_sentiment(review)
    print(f'Review : {review[:80]}...')
    print(f'Verdict: {label.upper()} ({confidence}% confidence)')
    print()
```

19. Final Summary

```python
print('=' * 55)
print('         IMDB SENTIMENT ANALYSIS - FINAL SUMMARY')
print('=' * 55)
print(f'Dataset Size       : {len(df):,} reviews')
print(f'Vocabulary Size    : {len(vectorizer.vocabulary_):,} features')
print(f'Train / Test Split : 67% / 33% (stratified)')
print()
print('Model Performances:')
for name, res in results.items():
    print(f'  {name:<25} CV={res["cv_mean"]:.4f}  Test={res["test_acc"]:.4f}')
print(f'  {"Voting Classifier":<25} Test={voting_acc:.4f}')
print()
print(f'Best Model         : Logistic Regression (GridSearchCV tuned)')
print(f'Best Test Accuracy : {accuracy_score(y_test, y_pred):.4f}')
print(f'ROC-AUC Score      : {roc_auc_score(y_test, y_pred_prob):.4f}')
print('=' * 55)
```

- **templates\index.html**

templates > index.html > ...

```html
<!doctype html>
<html lang="en">

<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>IMDB Sentiment App</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            background: #f6f7fb;
        }

        .container {
            max-width: 860px;
            margin: 40px auto;
            padding: 0 16px;
        }

        .card {
            background: white;
            border-radius: 12px;
            padding: 22px;
            box-shadow: 0 6px 18px rgba(0, 0, 0, 0.06);
        }

        textarea {
            width: 100%;
            min-height: 140px;
            padding: 12px;
            border-radius: 10px;
            border: 1px solid #d7d7e3;
            resize: vertical;
            font-size: 14px;
        }

        button {
            margin-top: 12px;
            padding: 10px 16px;
            border: none;
            border-radius: 10px;
            background: #4f46e5;
            color: white;
            font-weight: 700;
            cursor: pointer;
        }

        button:disabled {
            background: #a5a3ff;
            cursor: not-allowed;
        }

        .row {
            display: flex;
            gap: 14px;
            flex-wrap: wrap;
            align-items: center;
        }

        .status {
            margin-top: 14px;
            padding: 12px 14px;
            border-radius: 10px;
            background: #eef2ff;
            color: #3730a3;
            display: none;
        }

        .error {
            background: #fee2e2;
            color: #991b1b;
        }

        .muted {
            color: #6b7280;
            font-size: 13px;
        }

        .resultLabel {
            font-weight: 800;
            font-size: 18px;
        }
    </style>
</head>

```

```
<body>
    <div class="container">
        <div class="card">
            <h2>Sentiment Prediction (Positive / Negative)</h2>
            <p class="muted">Enter an IMDB-like review. The app will run the trained model and return the sentiment.</p>

            <textarea id="review" placeholder="Type review here..."></textarea>

            <div class="row">
                <button id="submitBtn" type="button">Submit</button>
            </div>

            <div id="status" class="status"></div>
        </div>
    </div>

    <script>
        const submitBtn = document.getElementById('submitBtn');
        const reviewEl = document.getElementById('review');
        const statusEl = document.getElementById('status');

        function showStatus(html, isError = false) {
            statusEl.innerHTML = html;
            statusEl.classList.toggle('error', isError);
            statusEl.style.display = 'block';
        }

        submitBtn.addEventListener('click', async () => {
            const text = (reviewEl.value || '').trim();
            if (!text) {
                showStatus('Please enter a review.', true);
                return;
            }

            submitBtn.disabled = true;
            showStatus('Predicting...', false);

            try {
                const resp = await fetch('/predict', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ review: text })
                });
```

```
                const data = await resp.json();

                if (!resp.ok) {
                    showStatus(data.error || 'Prediction failed', true);
                    return;
                }

                const label = data.label;
                const confidence = (data.confidence === null || data.confidence === undefined || Number.isNaN(data.confidence))
                    ? 'N/A'
                    : `${data.confidence}%`;

                showStatus(`<div class="resultLabel">${label.toUpperCase()}</div><div>Confidence: ${confidence}</div>`, false);
            } catch (e) {
                showStatus('Server error while predicting.', true);
            } finally {
                submitBtn.disabled = false;
            }
        });
    </script>
</body>

</html>
```

- **app.py**

```python
import os
import re
import joblib
import numpy as np
from flask import Flask, render_template, request, jsonify

# ---------- Load artifacts ----------
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "sentiment_model.pkl")
VECTORIZER_PATH = os.path.join(BASE_DIR, "tfidf_vectorizer.pkl")
ENCODER_PATH = os.path.join(BASE_DIR, "label_encoder.pkl")

model = joblib.load(MODEL_PATH)
vectorizer = joblib.load(VECTORIZER_PATH)
label_encoder = joblib.load(ENCODER_PATH)

# ---------- Minimal preprocessing (match training pipeline as closely as possible) ----------
# Your notebook uses NLTK stopwords + lemmatization. To avoid runtime downloads in Flask,
# we implement the same steps except lemmatization/stopword exactness.
# If you want exact matching, we can also package NLTK artifacts.
try:
    import nltk
    from nltk.corpus import stopwords
    from nltk.stem import WordNetLemmatizer

    _ = nltk.data.find("corpora/stopwords")
    _ = nltk.data.find("corpora/wordnet")

    _STOP_WORDS = set(stopwords.words("english"))
    _LEMMATIZER = WordNetLemmatizer()

    def clean_text(text: str) -> str:
        text = re.sub(r"<.*?>", " ", text)
        text = re.sub(r"http\S+|www\S+", "", text)
        text = text.lower()
        text = re.sub(r"[^a-z\s]", "", text)
        tokens = text.split()
        tokens = [_LEMMATIZER.lemmatize(w) for w in tokens if w not in _STOP_WORDS and len(w) > 2]
        return " ".join(tokens)

except Exception:

    # Fallback: lowercasing + strip HTML/URLs/punct, no stopword removal/lemmatization.
    def clean_text(text: str) -> str:
        text = re.sub(r"<.*?>", " ", text)
        text = re.sub(r"http\S+|www\S+", "", text)
        text = text.lower()
        text = re.sub(r"[^a-z\s]", "", text)
        tokens = text.split()
        tokens = [w for w in tokens if len(w) > 2]
        return " ".join(tokens)


def predict_sentiment(text: str):
    cleaned = clean_text(text)
    tfidf_vec = vectorizer.transform([cleaned])

    pred_int = model.predict(tfidf_vec)[0]
    # probability for the predicted class
    if hasattr(model, "predict_proba"):
        proba = model.predict_proba(tfidf_vec)[0][pred_int]
    else:
        proba = float("nan")

    label = label_encoder.inverse_transform([pred_int])[0]
    confidence = None if np.isnan(proba) else round(float(proba) * 100.0, 2)
    return label, confidence
```

```python

# ---------- Flask app ----------
app = Flask(__name__, template_folder="templates")


@app.route("/", methods=["GET"])
def index():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True) or {}
    review = (data.get("review") or "").strip()

    if not review:
        return jsonify({"error": "Review text is required"}), 400

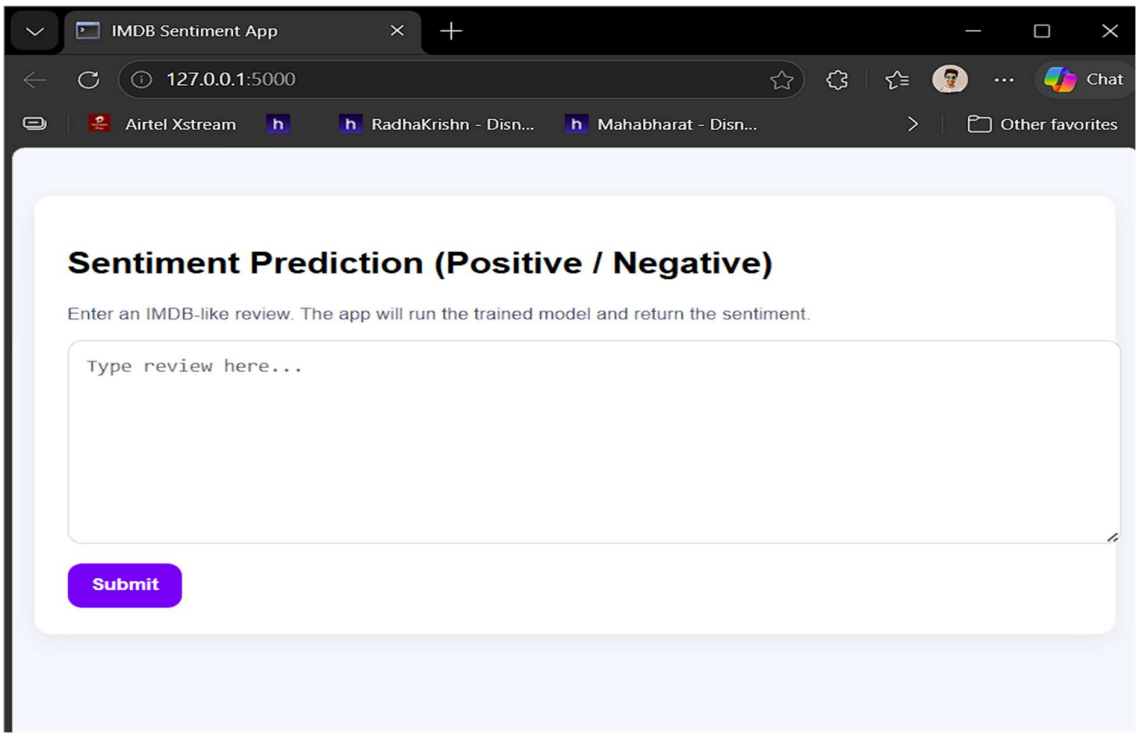
    label, confidence = predict_sentiment(review)
    return jsonify({"label": str(label), "confidence": confidence})


if __name__ == "__main__":
    # debug=True for development only
    app.run(host="127.0.0.1", port=5000, debug=True)


```

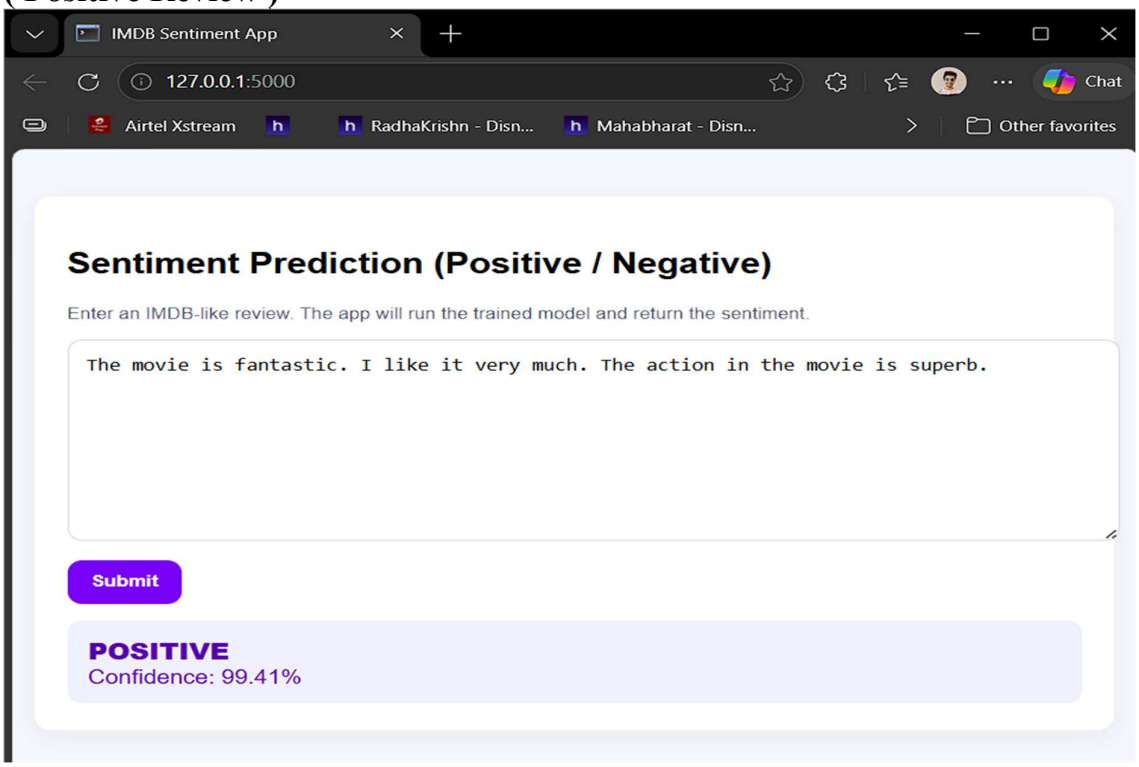
8. Screenshots of user interface or output

- **User Interface**

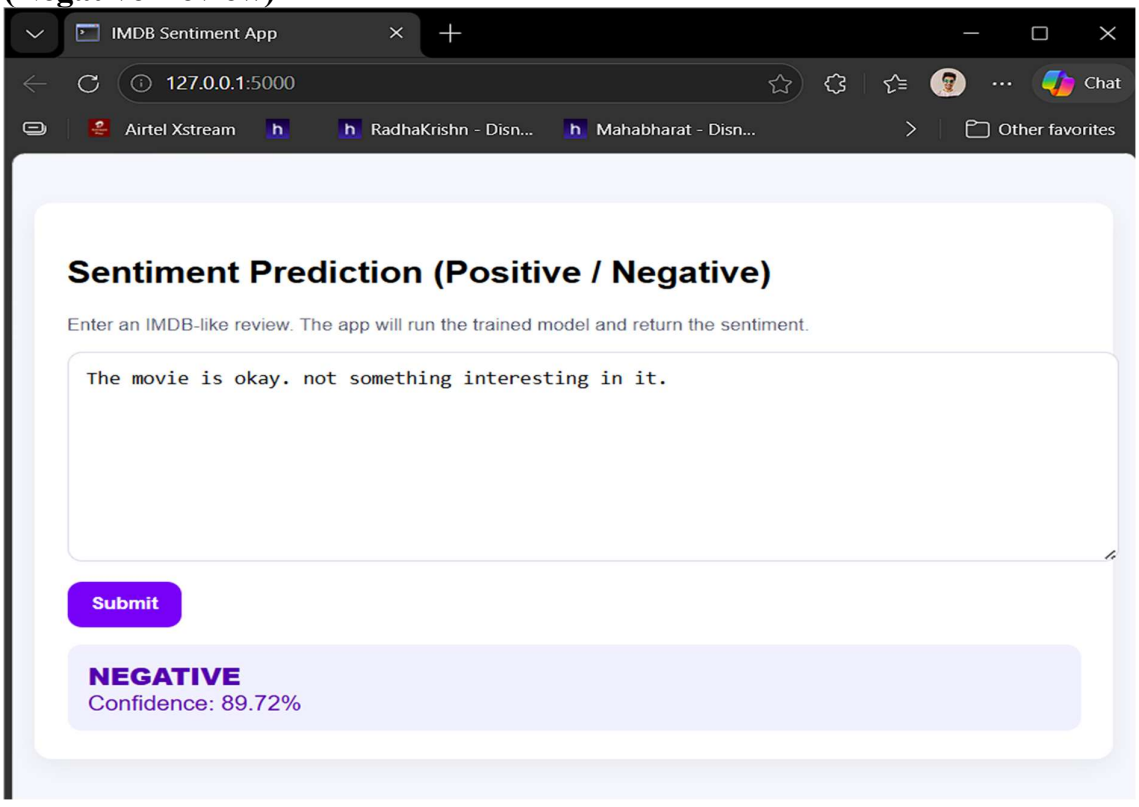


Output

(Positive Review)



(Negative Review)



9. Conclusion

The Sentiment Analysis project successfully demonstrates the application of machine learning and Natural Language Processing techniques in analyzing and classifying textual data. By implementing data preprocessing, feature extraction using TF-IDF, and a Logistic Regression model, the system is able to accurately predict whether a movie review expresses a positive or negative sentiment. The integration of a Flask-based web interface further enhances the usability of the system by allowing users to input their own reviews and receive real-time predictions, making the project practical and user-friendly.

Overall, the project highlights the importance of automated text analysis in handling large volumes of data efficiently and accurately. It overcomes the limitations of traditional manual and rule-based approaches by providing a scalable and reliable solution. Although the current system performs well, it can be further improved by incorporating advanced techniques such as deep learning models, multilingual support, and enhanced context understanding. This project serves as a strong foundation for future developments in sentiment analysis and real-world applications involving opinion mining.